SR-MPLS Training Lab

M3 L3VPN Services

Estimated time: 75 min

Objective

Turn the SR-MPLS transport you built in M2 into a real customer service. You will stand up the iBGP overlay to the C1 and C2 route reflectors with family inet-vpn, build the CUST_A and CUST_B VRFs on the PEs, run eBGP to the customer edge routers, and prove that Linux Client1 behind CE1 reaches Linux Client2 behind CE2 entirely inside CUST_A. You will then prove that CUST_A and CUST_B stay completely isolated even though both customers advertise the identical 192.168.50.0/24 prefix.

Before you start: M2 complete: SR-MPLS transport is up and every remote PE loopback resolves to a Push label in inet.3, so the overlay has working tunnels to ride on.

Concept primer

An L3VPN separates two planes of routing. The transport (underlay) you already have carries packets between PE loopbacks over SR label-switched paths. The overlay is a single iBGP mesh that carries the customer routes on top of that transport. Rather than peer every PE to every other PE, each PE runs one iBGP session to C1 and one to C2, the two core route reflectors, and enables 'family inet-vpn unicast' so the session carries VPN-IPv4 (labelled L3VPN) routes. The reflectors re-advertise each PE's VPN routes to all the others, so adding a PE never touches the existing ones. The PE next-hop on every VPN route is a remote PE loopback, which resolves through inet.3 to an SR tunnel; that is the exact handoff between the overlay and the transport you built in M2.

Two BGP communities do the heavy lifting, and new engineers constantly confuse them. The route distinguisher (RD) is an 8-byte value prepended to each customer prefix to make it globally unique in the bgp.l3vpn.0 table. CUST_A on PE1 uses RD 10.0.0.1:100, so CE1's 192.168.50.1/32 is advertised as 10.0.0.1:100:192.168.50.1/32. The RD does no filtering at all; it only guarantees that CUST_A's 192.168.50.0/24 and CUST_B's identical 192.168.50.0/24 are two distinct NLRI and never overwrite each other in BGP. The route target (RT) is a separate extended community that

acts as the import/export filter. CUST_A tags its routes with target:64512:100 on export and imports only that value; CUST_B uses target:64512:200. A PE installs a received VPN route into a VRF only when the route's RT matches that VRF's import target. RD makes prefixes unique; RT decides which VRF they land in.

'vrf-table-label' tells the PE to allocate one MPLS label per VRF and to perform an IP lookup in that VRF's table after popping the label, which is what lets a single label serve every prefix in the VRF. On the customer side each VRF runs eBGP to its CE (PE AS 64512, CE AS 65001) over a VLAN subinterface of ge-0/0/3. Because the RTs differ, a route that CE1 sends into CUST_A can never leak into CUST_B, so both customers safely reuse 192.168.50.0/24. Client1 and Client2 both live in CUST_A on distinct subnets (10.50.1.0/24 and 10.50.2.0/24), so their traffic crosses the core as CUST_A VPN routes; the overlapping 192.168.50.1 loopbacks exist once in each VRF and are only reachable from inside their own VRF, which is the isolation you will demonstrate.

Build steps

1. **PE1: build the iBGP overlay to both route reflectors with the inet-vpn family.**

```
set protocols bgp group OVERLAY type internal
set protocols bgp group OVERLAY local-address 10.0.0.1
set protocols bgp group OVERLAY family inet-vpn unicast
set protocols bgp group OVERLAY family evpn signaling
set protocols bgp group OVERLAY neighbor 10.0.0.11
set protocols bgp group OVERLAY neighbor 10.0.0.12
```

Note: type internal makes this iBGP inside AS 64512. local-address ties the session to the loopback so it follows the SR transport. The two neighbors are C1 (10.0.0.11) and C2 (10.0.0.12), the route reflectors. family inet-vpn unicast is what upgrades this from a plain iBGP session to an L3VPN session.

2. **PE1: create the CUST_A VRF, its RD, RT, table label, subinterface, and eBGP session to CE1.**

```
set interfaces ge-0/0/3 vlan-tagging
set interfaces ge-0/0/3 encapsulation flexible-ethernet-services
set interfaces ge-0/0/3 unit 100 vlan-id 100
set interfaces ge-0/0/3 unit 100 family inet address 172.16.100.1/30
set routing-instances CUST_A instance-type vrf
set routing-instances CUST_A interface ge-0/0/3.100
set routing-instances CUST_A route-distinguisher 10.0.0.1:100
set routing-instances CUST_A vrf-target target:64512:100
```

```
set routing-instances CUST_A vrf-table-label
set routing-instances CUST_A protocols bgp group CE type external
set routing-instances CUST_A protocols bgp group CE peer-as 65001
set routing-instances CUST_A protocols bgp group CE neighbor 172.16.100.2
```

Note: instance-type vrf creates the CUST_A.inet.0 table. The RD 10.0.0.1:100 makes CUST_A prefixes globally unique; the RT target:64512:100 is the import/export filter. peer-as 65001 is the customer AS; the PE-CE link is eBGP over VLAN 100.

3. **PE1: create the CUST_B VRF the same way with its own RD and RT on VLAN 200.**

```
set interfaces ge-0/0/3 unit 200 vlan-id 200
set interfaces ge-0/0/3 unit 200 family inet address 172.16.101.1/30
set routing-instances CUST_B instance-type vrf
set routing-instances CUST_B interface ge-0/0/3.200
set routing-instances CUST_B route-distinguisher 10.0.0.1:200
set routing-instances CUST_B vrf-target target:64512:200
set routing-instances CUST_B vrf-table-label
set routing-instances CUST_B protocols bgp group CE type external
set routing-instances CUST_B protocols bgp group CE peer-as 65001
set routing-instances CUST_B protocols bgp group CE neighbor 172.16.101.2
commit
```

Note: CUST_B is identical in structure but RD 10.0.0.1:200 and RT target:64512:200 keep it a separate VPN. This is the deliberate second customer that reuses 192.168.50.0/24; the different RT is the only thing keeping it isolated from CUST_A.

4. **C1 (core route reflector): enable the overlay with a cluster ID and peer the PEs and the other reflector.**

```
set protocols bgp group OVERLAY type internal
set protocols bgp group OVERLAY local-address 10.0.0.11
set protocols bgp group OVERLAY family inet-vpn unicast
set protocols bgp group OVERLAY family evpn signaling
set protocols bgp group OVERLAY cluster 0.0.0.1
set protocols bgp group OVERLAY neighbor 10.0.0.1
set protocols bgp group OVERLAY neighbor 10.0.0.2
set protocols bgp group OVERLAY neighbor 10.0.0.12
commit
```

Note: cluster 0.0.0.1 makes C1 a route reflector; the PE neighbors (10.0.0.1 PE1, 10.0.0.2 PE2) become clients and their VPN routes are reflected to each other. 10.0.0.12 is C2, the redundant reflector. C2 mirrors this config with local-address 10.0.0.12. The reflectors carry no VRF; they only relay VPN NLRI.

5. **CE1 (customer edge): run eBGP up to PE1 and advertise the customer prefixes from a virtual-router instance.**

```
set routing-instances CUST_A instance-type virtual-router
set routing-instances CUST_A interface ge-0/0/1.100
set routing-instances CUST_A interface lo0.100
set interfaces lo0 unit 100 family inet address 192.168.50.1/32
set interfaces ge-0/0/2 unit 0 family inet address 10.50.1.1/24
set routing-instances CUST_A interface ge-0/0/2.0
set routing-instances CUST_A protocols bgp group TO-PE type external
set routing-instances CUST_A protocols bgp group TO-PE peer-as 64512
set routing-instances CUST_A protocols bgp group TO-PE neighbor 172.16.100.1
set routing-instances CUST_A protocols bgp group TO-PE export ADV-CUST
set routing-options autonomous-system 65001
set policy-options policy-statement ADV-CUST term lo from protocol direct
set policy-options policy-statement ADV-CUST term lo then accept
commit
```

Note: The CE has no MPLS or VPN awareness; it is a plain eBGP speaker. instance-type virtual-router gives it a private routing table. ge-0/0/2 (10.50.1.1/24) is Client1's gateway, and ADV-CUST accepts the direct routes (the 10.50.1.0/24 client subnet and the 192.168.50.1/32 loopback) so PE1 learns them over eBGP. CE1 also builds a CUST_B virtual-router on VLAN 200 the same way, reusing 192.168.50.1/32 as lo0.200.

6. **Or push the whole service tier at once.**

```
python push_lab_configs.py stage3-l3vpn
```

Note: Each node's overlay, VRF, RR, or CE lines live in its own configs/stage3-l3vpn/<node>.set. PE1/PE2 get VRFs, C1/C2 get the reflector config, CE1/CE2 get the customer edge config. Push per node from those files or run the whole stage in one shot.

Verification

show bgp summary

```
Groups: 2 Peers: 4 Down peers: 0
Table          Tot Paths  Act Paths Suppressed  History Damp State Pending
bgp.l3vpn.0           6         6         0       0        0      0
Peer           AS     InPkt  OutPkt  Flaps Last Up/Dwn State|#Active/Received/Accepted
10.0.0.11     64512       412     410     0    3:01:22 Establ
  bgp.l3vpn.0: 3/3/3/0
10.0.0.12     64512       408     407     0    3:01:19 Establ
  bgp.l3vpn.0: 3/3/3/0
172.16.100.2  65001       210     212     0    2:58:40 Establ
```

```
 CUST_A.inet.0: 2/2/2/0
172.16.101.2  65001       205     207      0    2:58:33 Establ
 CUST_B.inet.0: 1/1/1/0
```

What to look for: The two iBGP sessions to the reflectors C1 (10.0.0.11) and C2 (10.0.0.12) are Establ and feed bgp.l3vpn.0. The two eBGP sessions to CE1 (172.16.100.2 in CUST_A) and CE2's side (172.16.101.2 in CUST_B) are Establ inside their VRFs. If a reflector session is not Establ, no remote VPN routes arrive and every VRF looks empty.

show route table CUST_A.inet.0

```
CUST_A.inet.0: 5 destinations, 5 routes (5 active, 0 holddown, 0 hidden)
+ = Active Route, - = Last Active, * = Both

10.50.1.0/24    *[BGP/170] 02:58:40, localpref 100
                  AS path: 65001 I, from 172.16.100.2
                >  to 172.16.100.2 via ge-0/0/3.100
10.50.2.0/24    *[BGP/170] 02:55:10, localpref 100, from 10.0.0.2
                  AS path: 65001 I
                >  to 10.1.1.2 via ge-0/0/1.0, Push 16002, Push 299872(top)
172.16.100.0/30 *[Direct/0] 03:02:00
                >  via ge-0/0/3.100
192.168.50.1/32 *[BGP/170] 02:58:40, localpref 100
                  AS path: 65001 I, from 172.16.100.2
```

What to look for: 10.50.1.0/24 (Client1's subnet) is learned locally from CE1 over eBGP. 10.50.2.0/24 (Client2's subnet) is the remote VPN route: its next-hop is PE2 (10.0.0.2) and it carries a two-label stack, the VPN label 299872 over the SR transport label Push 16002. That outer label is the M2 tunnel doing its job under the overlay.

show route receive-protocol bgp 10.0.0.11 table bgp.l3vpn.0

```
bgp.l3vpn.0: 6 destinations, 6 routes (6 active, 0 holddown, 0 hidden)
 Prefix                  Nexthop     Lclpref  AS path
 10.0.0.2:100:10.50.2.0/24     10.0.0.2    100      65001 I
 10.0.0.2:100:192.168.50.1/32  10.0.0.2    100      65001 I
 10.0.0.2:200:192.168.50.1/32  10.0.0.2    100      65001 I
```

What to look for: These are PE2's VPN routes reflected by C1. Note how the RD prefix (10.0.0.2:100 for CUST_A, 10.0.0.2:200 for CUST_B) makes the two identical 192.168.50.1/32 prefixes distinct NLRI, so they coexist in one table. Which VRF each lands in is decided later by the RT community, not by the RD.

Client1 -> Client2 reachability, then CUST_A / CUST_B isolation

```
client1:~$ ping -c 3 10.50.2.10
PING 10.50.2.10 (10.50.2.10) 56(84) bytes of data.
64 bytes from 10.50.2.10: icmp_seq=1 ttl=61 time=3.42 ms
64 bytes from 10.50.2.10: icmp_seq=2 ttl=61 time=2.91 ms
64 bytes from 10.50.2.10: icmp_seq=3 ttl=61 time=2.88 ms
3 packets transmitted, 3 received, 0% packet loss

lab@PE1> ping routing-instance CUST_A 192.168.50.1 count 2 rapid
!!  round-trip min/avg/max = 1.9/2.1/2.3 ms   (reaches CE1 CUST_A loopback)
lab@PE1> ping routing-instance CUST_B 192.168.50.1 count 2 rapid
!!  round-trip min/avg/max = 1.8/2.0/2.2 ms   (reaches CE1 CUST_B loopback)
```

What to look for: End to end: Client1 (10.50.1.10) reaches Client2 (10.50.2.10) across the core inside CUST_A, ttl 61 showing the four router hops CE1-PE1-PE2-CE2. The two pings to the identical 192.168.50.1 both succeed but land on different loopbacks because each stays in its own VRF. Same address, two customers, zero leakage: that is the isolation the RTs enforce.

[screenshot 004: M3 verification: CUST_A client-to-client reachability and CUST_B isolation]

Break / Fix exercise

Scenario: The service is cut over and CUST_A is working: Client1 and Client2 ping each other cleanly. CUST_B is a different story. Both PEs report every BGP session Up, the SR transport is healthy, and nothing in the logs looks wrong, yet CUST_B reachability across the core fails. When you look in bgp.l3vpn.0 the remote CUST_B prefixes are clearly present and active, but they never appear in the CUST_B.inet.0 VRF table on the affected PE, so that customer's traffic has nowhere to go.

Your task: Explain why the routes can sit in bgp.l3vpn.0 while every BGP session stays Up, then find why they are not being installed into CUST_B.inet.0 and restore CUST_B reachability. Confirm CUST_A is untouched throughout, and prove the fix by installing the remote CUST_B route into the VRF and pinging across the core.

Hint: A route reaching bgp.l3vpn.0 but not the VRF is an import decision, not a session or transport problem, which is why nothing is Down. The thing that decides whether a VPN route enters a given VRF is an extended community, and it must match on both the sending and receiving side. Compare what the CUST_B routes carry in bgp.l3vpn.0 against what the CUST_B VRF is configured to import on the broken PE, and remember the healthy PEs are the reference for the correct value.

The instructor holds the fault-injection and fix commands in the answer key. Diagnose it yourself first.

Brief it back

Explain each of these aloud before moving on. If you cannot, re-read the primer and re-run the verification commands.

- Explain the difference between a route distinguisher and a route target, and describe exactly how each one contributes to letting CUST_A and CUST_B both use 192.168.50.0/24 without conflict.
- During a route-target mismatch on one VRF, why do all BGP sessions stay Up and why do the affected routes still show up in bgp.l3vpn.0, even though the VRF table and customer pings are broken?
- Trace the full control-plane path of a single CUST_A customer route from CE1 to CE2: how it enters PE1, what RD and RT and labels get attached, how the route reflectors move it, and how PE2 decides to install it in CUST_A.inet.0 and hand it to CE2.